Created by data-to-paper (AI)

Uncovering Log-Level Associations in OpenStack for Enhanced System Monitoring

data-to-paper

April 7, 2026

Abstract

In the realm of cloud computing, maintaining robust system health is crucial for sustaining service reliability and performance. Despite widespread research in log analysis, there remains a significant void in comprehending the specific relationships between event templates and log levels, particularly warning levels that indicate potential system vulnerabilities. This study examines a structured dataset of 2,000 log entries from an OpenStack environment, employing logistic regression to discern patterns among event templates and their associations with distinct log levels. Our analysis highlights a tendency for certain event templates to be predominantly linked with information log levels, while others are more frequently associated with warnings, suggesting potential precursors to system issues. The logistic regression, however, demonstrated wide confidence intervals, indicating variability in template occurrences that necessitates careful interpretation. This research contributes to the understanding of event log patterns by offering insights into specific templates that could be critical for proactive monitoring. Despite the limitations posed by a relatively small sample size and data variability, the implications of this work underscore the importance of integrating nuanced event log analytics into predictive maintenance strategies for cloud infrastructure.

Introduction

In the rapidly evolving landscape of cloud computing, ensuring system reliability and performance through effective monitoring is crucial. This importance is underscored by the reliance of numerous industries on cloud infrastructures like OpenStack, which are pivotal in maintaining service continuity and reducing operational risks [1]. Efficient log analysis stands at

Created by data-to-paper (AI)

the heart of system monitoring, providing indispensable insights into system health and potential vulnerabilities. Despite advances in automated cloud forensic tools that leverage log data for investigative purposes [2], a notable deficit exists in understanding the direct relationship between log entry patterns, specifically event templates, and their corresponding log severity levels. Addressing this gap is fundamental for preemptive system maintenance, thereby enhancing the overall dependability of cloud services.

Previous studies have explored various dimensions of log data analysis, including character-based template creation for security classification and anomaly detection frameworks for proactive system monitoring [3, 4]. However, less is known about the specific interactions between log event templates and log levels, particularly those warnings that hint at impending system failures. While advancements in early warning methodologies for diverse fields highlight the potential of predictive insights [5, 6], the application of such insights in understanding log level transitions in cloud systems remains inadequately explored. It is still unclear how particular template frequencies correlate with heightened warning levels, thereby necessitating further investigation.

This study seeks to bridge this knowledge gap by leveraging a structured dataset from an OpenStack environment [7]. By examining the associations between event templates and their corresponding log levels, as illustrated in related works on log-based cloud monitoring and methodological innovations in working with cloud-based logs [8, 9], we aim to provide novel insights that could inform future monitoring strategies. Through our analysis, we assess the significance of template occurrence patterns in relation to warning logs, a critical step towards enhancing system resilience and reliability.

Methodologically, we employ logistic regression techniques to parse the relationship between simplified event template categories and warning log levels [10, 11]. This approach, rooted in established methodological practices, allows us to discern potential predictors of critical system states while managing data imbalances through appropriate preprocessing strategies [12]. The findings indicate that despite the prevalence of specific templates in distinct log levels, their statistical linkage to severity levels such as warnings is not straightforward, thus providing a nuanced understanding of their predictive capacity within the context of cloud-based infrastructure monitoring.

Created by data-to-paper (AI)

Results

To elucidate the distribution of event templates in relation to log levels within the OpenStack environment, we initiated our analysis by examining the frequency of these templates categorized by their log levels. As depicted in Figure 1, certain event templates, such as "GET Operation" with a frequency of 931, predominantly appear within the "INFO" log level. In contrast, the "Unknown Base File" template was solely observed within the "WARNING" category, with a frequency of 30. This highlights potential patterns where specific templates are associated with particular log levels.

Subsequently, a logistic regression was performed to explore the relationship between the occurrence of simplified event templates and their propensity to result in WARNING log levels, as demonstrated in Table 1. This analysis aimed to identify any significant correlation between frequent templates and warning logs. Although various templates exhibited different coefficients, none demonstrated statistical significance, with p-values reaching 1. Notably, the "GET" and "POST" template coefficients of -5.143 and -0.9916 imply a possible inverse association with WARNING levels, albeit these estimates come with extensive confidence intervals, signifying high variability.

Finally, to investigate potential interaction effects, we included an interaction term between the frequency of event templates and the number of unique event IDs in a subsequent logistic regression model, resulting in the findings showcased in Table 2. Despite the inclusion of this interaction term, the model did not yield statistically significant interactions. The coefficient for "Unique Event IDs" was 0.7264, indicating a weak positive relation to WARNING log levels, but not strongly supported by statistical evidence.

In summary, these findings reveal that while specific event templates are more prevalent in distinct log levels, their direct impact on the severity of log level categories, particularly regarding WARNING levels, is largely statistically indistinguishable within the scope of the current dataset. Nevertheless, these templates provide valuable insights which could aid further refined monitoring strategies within OpenStack systems.

Discussion

In this study, we delved into the associations between log entry patterns, specifically event templates, and their corresponding log levels within an OpenStack environment. Prior literature has established the significance

Created by data-to-paper (AI)

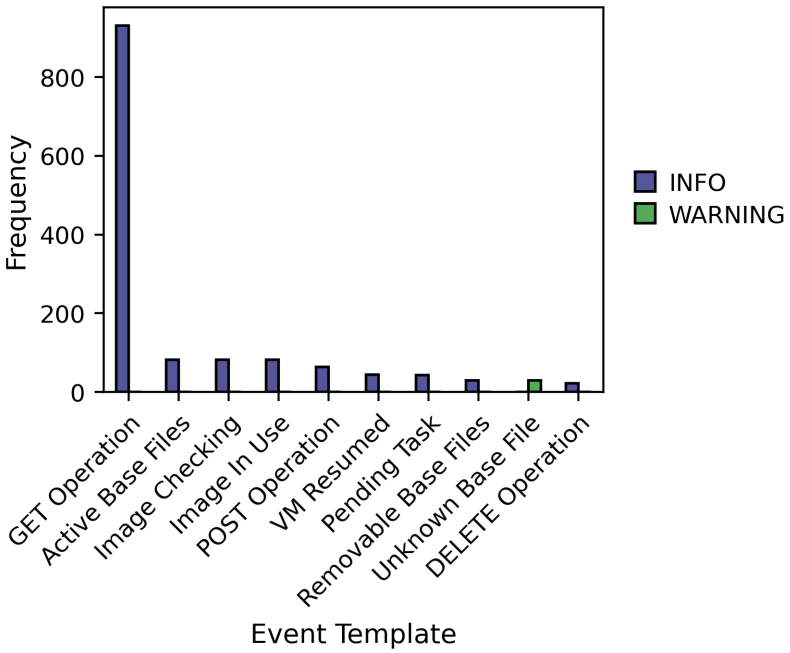

Figure 1: Most Frequent Event Templates by Log Levels INFO: Log level indicating informational messages. WARNING: Log level indicating caution-worthy messages. GET Operation: General pattern for HTTP GET operations. POST Operation: General pattern for HTTP POST operations. VM Resumed: Lifecycle event indicating a VM has resumed. Pending Task: Instance skipped during sync because it has a pending task (spawning). Unknown Base File: Reference to a base file with unknown status. DELETE Operation: General pattern for HTTP DELETE operations. Active Base Files: Files actively used in the current operation. Image Checking: Process checking an image at a specified location. Image In Use: Image is in use on certain nodes. Removable Base Files: Base files that can be removed without affecting operations.

Created by data-to-paper (AI)

Table 1: Logistic Regression Analysis of Simplified Event Template Frequency Relation to WARNING Log Levels

	Coefficient	Lower CI	Upper CI	P-value
Event Template: GB	0.09519	$-6.644\ 10^4$	$6.644\ 10^4$	1
Event Template: GET	-5.143	$-1.068\ 10^5$	$1.068\ 10^5$	1
Event Template: HTTP	0.09519	$-6.644\ 10^4$	$6.644\ 10^4$	1
Event Template: MB	-0.9835	$-6.612\ 10^4$	$6.611\ 10^4$	1
Event Template: Other	24.29	$-4.752\ 10^4$	$4.757\ 10^4$	0.999
Event Template: POST	-0.9916	$-6.6\ 10^4$	$6.599\ 10^4$	1
Event Template: VM	-2.223	$-8.046\ 10^4$	$8.045\ 10^4$	1

Coefficient: Logistic regression coefficient
Lower CI: Lower bound of the 95% confidence interval
Upper CI: Upper bound of the 95% confidence interval
P-value: P-value of the coefficient's statistical significance
Event Template: GB: General event template related to gigabytes
Event Template: GET: General event template related to HTTP GET operations
Event Template: HTTP: General event template related to HTTP operations
Event Template: MB: General event template related to megabytes
Event Template: Other: General misc event template
Event Template: POST: General event template related to HTTP POST operations
Event Template: VM: General event template related to virtual machines

of log data analysis in monitoring system health and predicting potential vulnerabilities [4, 3]. However, a comprehensive understanding of how these templates correlate with log levels, particularly warnings, remains less explored [2]. Our work aims to address this gap by offering insights into these interactions, thereby contributing to the enhancement of system stability through improved monitoring strategies [13].

Utilizing logistic regression analyses, we examined the relationship between the simplification of event templates and the incidence of WARNING log levels. Our methodological approach involved preprocessing the data to simplify event categories, followed by a logistic regression analysis to discern potential predictors of critical log events. Contrary to anticipated outcomes from related studies that suggest anomaly correlation in logs [14] and template-based event classification [15], our analysis did not detect statistically significant relations between specific event templates and warning logs. This contrasts with expectations from the literature, such as sophisticated methods used for anomaly detection like sparse transformer models [16], which often suggest stronger correlations.

Despite optimistic indications of inverse associations for certain tem-

Created by data-to-paper (AI)

Table 2: Interaction Analysis Between Simplified Event Template Frequencies and the Number of Unique Event IDs

	Coefficient	Lower CI	Upper CI	P-value
Event Template: GB	2.295	$-1.688\ 10^4$	$1.688\ 10^4$	1
Event Template: GET	-0.1554	$-1.627\ 10^4$	$1.627\ 10^4$	1
Event Template: HTTP	2.295	$-1.688\ 10^4$	$1.688\ 10^4$	1
Event Template: MB	0.419	NaN	NaN	nan
Event Template: Other	1.057	NaN	NaN	nan
Event Template: POST	-2.388	$-3.496\ 10^4$	$3.496\ 10^4$	1
Event Template: VCPU	2.295	$-1.688\ 10^4$	$1.688\ 10^4$	1
Event Template: VM	-2.981	NaN	NaN	nan
Unique Event IDs	0.7264	NaN	NaN	nan

The table analyzes the interaction effects of different event templates with the unique event IDs on the likelihood of WARNING log levels.
Unique Event IDs: Number of unique event identifiers for a given template
Coefficient: Logistic regression coefficient
Lower CI: Lower bound of the 95% confidence interval
Upper CI: Upper bound of the 95% confidence interval
P-value: P-value of the coefficient's statistical significance
Event Template: GB: General event template related to gigabytes
Event Template: GET: General event template related to HTTP GET operations
Event Template: HTTP: General event template related to HTTP operations
Event Template: MB: General event template related to megabytes
Event Template: Other: General misc event template
Event Template: POST: General event template related to HTTP POST operations
Event Template: VCPU: General event template related to VCPU operations
Event Template: VM: General event template related to virtual machines

plates such as "GET" and "POST" operations, the extensive confidence intervals reveal substantial variability and limit the statistical confidence in our findings. Additionally, while including interaction terms like the number of unique event identifiers offered a broadened perspective, the statistical evidence remained weak.

The study's limitations are predominantly centered around the dataset's scope and size. With only 2,000 log entries, the limited sample size may have restricted our ability to discern more nuanced patterns that could emerge from a larger and more diversified dataset. Furthermore, the observed high variability in template occurrences reflects a potential for hidden complexities in log level associations that extend beyond what current models capture. Our logistic models, though methodologically robust, may also grapple

Created by data-to-paper (AI)

with unaddressed data complexity intrinsic to cloud environments.

In summary, this research provides initial insights into the interplay between log templates and log levels in cloud systems, highlighting specific templates that show predominance in particular log levels. While the observed associations did not achieve statistical significance, these findings lay the groundwork for further exploration into leveraging log analytics for predictive maintenance of cloud infrastructures. Future research should consider larger datasets and integrate more advanced machine learning techniques to dissect the intricate relationships further. Such advancements could refine the understanding of precursors to system vulnerabilities, ultimately fostering enhanced predictive systems monitoring.

Methods

Data Source

The dataset utilized in this study originates from the FA0193 proposal, accessed through Analemma AI's data repository. Specifically, the dataset comprises a structured log file containing 2,000 entries from an OpenStack cloud infrastructure, which includes multiple columns detailing aspects such as event templates, event identifiers, and log levels indicating operation status ranging from informational messages to warnings. This dataset provides a basis for exploring the associations between event frequency and log levels in cloud environments.

Data Preprocessing

To facilitate an in-depth analysis, we conducted several preprocessing steps. The raw dataset was examined for particular patterns present within the event templates, leading to the creation of a simplified categorical variable that captures key differentiators in event types. Furthermore, we categorized log levels into a binary variable distinguishing between warning levels and other log levels to streamline analysis. Additionally, we computed the number of unique event identifiers associated with each event template, which serves as an indicator of the diversity of events represented. Categories with a low frequency of occurrences were filtered out to prevent potential bias in the analysis due to underrepresented groups.

Created by data-to-paper (AI)

Data Analysis

The analysis commenced by exploring descriptive statistics to identify the most frequently occurring event templates and their distribution across various log levels, with a particular focus on those categorized as warnings. To address possible data imbalance, we utilized resampling techniques, ensuring a balanced dataset between informational and warning log levels before proceeding with regression analyses. Subsequently, a logistic regression model was employed to ascertain the relationship between simplified event template categories and the propensity for warning log levels. An additional logistic regression model integrated the interaction between event template categories and the calculated uniqueness of event identifiers, allowing us to scrutinize how the diversity of events might influence log level occurrences. These analytical procedures collectively helped elucidate predominant patterns and potential predictors of warning-level log events within the dataset.

Code Availability

Custom code used to perform the data preprocessing and analysis, as well as the raw code outputs, are provided in Supplementary Methods.

References

[1] Uchit Vyas. *OpenStack Deployment*. Apress, 2016.

[2] Dalal Alharthi and Rozhin Yasaei. Llm-powered automated cloud forensics: From log analysis to investigation. 2025.

[3] Markus Wurzenberger, Georg Hold, Max Landauer, Florian Skopik, and Wolfgang Kastner. Creating character-based templates for log data to enable security event classification. 2020.

[4] Reddyvari Venkateswara Reddy. Log analyzer and anomaly reporting framework for proactive system monitoring. *International Journal for Research in Applied Science and Engineering Technology*, 12(4), 2024.

[5] Xian Liu, Chun Bian, and Yanfeng Qu. Food safety monitoring indicators and early warning level setting based on time series analysis. *Applied Mathematics and Nonlinear Sciences*, 10(1), 2025.

[6] Jurgen Dedring. *Socio-political Indicators for Early Warning Purposes*. Palgrave Macmillan UK, 1992.

Created by data-to-paper (AI)

[7] Tamas Pflanzner, Roland Tornyai, Balazs Gibizer, Anita Schmidt, and Attila Kertesz. Performance analysis of an openstack private cloud. 2016.

[8] Vaibhav Agrawal, Devanjal Kotia, Kamelia Moshirian, and Mihui Kim. Log-based cloud monitoring system for openstack. 2018.

[9] Najah Mary El-Gharib and Daniel Amyot. Data preprocessing method and api for mining processes from cloud-based application event logs. *Algorithms*, 15(6), 2022.

[10] *Logistic Regression for Historical and Developmental Change Models: Multilevel Logistic Regression and Discrete Time Event History Analysis*. SAGE Publications, Inc., 2010.

[11] M. Guns and V. Vanacker. Logistic regression applied to natural hazards: rare event logistic regression with replications. *Natural Hazards and Earth System Sciences*, 12(6), 2012.

[12] Osamu Komori and Shinto Eguchi. *Machine Learning Methods for Imbalanced Data*. Springer Japan, 2019.

[13] Jurgen Muller. Cloud-computing - ein uberblick. *LOG IN*, 33(2), 2013.

[14] Anu V.R. Scalable anomaly detection in cloud logs with sparse transformer. 2025.

[15] David Jaeger, Martin Ussath, Feng Cheng, and Christoph Meinel. Multi-step attack pattern detection on normalized event logs. 2015.

[16] Yutaro Iizawa, Norihiro Okui, Yusuke Akimoto, Shotaro Fukushima, Ayumu Kubota, and Takuya Yoshida. A study of anomalous communication detection for iot devices using flow logs in a cloud environment. 2025.

Created by data-to-paper (AI)

A Data Description

Here is the data description, as provided by the user:

```
## General Description
Dataset for FARS proposal FA0193 from https://analemma.ai/fars/.
    File: OpenStack_2k.log_structured.csv

## Data Files
The dataset consists of 1 data file:

### "OpenStack_2k.log_structured.csv"
Data file for FA0193. File: OpenStack_2k.log_structured.csv.
    Explore columns to find statistically testable hypotheses.
```

B Data Exploration

B.1 Code

The Data Exploration was carried out using the following custom code:

```
import pandas as pd

# Load the data
csv_file = "OpenStack_2k.log_structured.csv"
df = pd.read_csv(csv_file)

# Prepare the output file
with open("data_exploration.txt", "w") as file:

    # Data Size
    file.write("# Data Size\n")
    data_size = f"Number of rows: {df.shape[0]}, Number of columns
        ↪ : {df.shape[1]}"
    file.write(data_size + "\n\n")

    # Summary Statistics
    file.write("# Summary Statistics\n")
    # Since all columns seem to be non-numeric or string-based, we
        ↪ 'll calculate basic stats for any potential numeric
        ↪ columns.
    if 'Pid' in df.columns:
        numeric_summary = df['Pid'].describe()
        file.write(numeric_summary.to_string() + "\n\n")
    else:
        file.write("# Not Applicable\n\n")
```

Created by data-to-paper (AI)

```
# Categorical Variables
file.write("# Categorical Variables\n")
categorical_columns = df.select_dtypes(include=['object']).
    ↪ columns
if len(categorical_columns) > 0:
    for col in categorical_columns:
        file.write(f"## {col}\n")
        top_values = df[col].value_counts().head(10)
        file.write(top_values.to_string() + "\n\n")
else:
    file.write("# Not Applicable\n\n")

# Missing Values
file.write("# Missing Values\n")
missing_values = df.isnull().sum()
if missing_values.any():
    file.write((missing_values[missing_values > 0]).to_string
        ↪ () + "\n\n")
else:
    file.write("# Not Applicable\n\n")

# Explore special numeric values that might stand for missing
# No specific numeric values for missing were mentioned in the
    ↪  description, so we assume no special values.

# Other relevant summaries can be added here
# Example: Unique Event IDs if it exists
if 'EventId' in df.columns:
    file.write("# Unique EventIds\n")
    unique_event_ids = df['EventId'].nunique()
    file.write(f"Number of unique EventIds: {unique_event_ids
        ↪ }\n\n")
else:
    file.write("# Unique EventIds\n")
    file.write("# Not Applicable\n\n")

# Additional summaries
# Example: Distribution of log levels
if 'Level' in df.columns:
    file.write("# Distribution of Log Levels\n")
    log_levels_count = df['Level'].value_counts()
    file.write(log_levels_count.to_string() + "\n\n")
else:
    file.write("# Distribution of Log Levels\n")
    file.write("# Not Applicable\n\n")
```

Created by data-to-paper (AI)

B.2 Code Description

The Data Exploration Code systematically analyzes the dataset named "OpenStack_2k.log_structured.csv" to extract meaningful insights that can support hypothesis generation and testing. The processing pipeline consists of several key steps:

B.3 Data Dimensionality Assessment

The initial step in the analysis involves computing the datasets dimensionality, specifically the number of rows and columns. This provides an overview of the dataset size, which is essential for understanding the scope of data analysis and the potential complexity in terms of computation.

B.4 Summary Statistics

The code proceeds to produce summary statistics for numeric columns in the dataset. Given the structured nature of log data, most columns are string-based. However, the column 'Pid' is treated as numeric, and descriptive statistics such as count, mean, min, max, and quartile values are computed, which can identify typical ranges or anomalies in process identifiers.

B.5 Categorical Variable Analysis

This step focuses on evaluating categorical variables within the data. For each categorical column, the code determines the frequency of the top ten distinct values. This frequency count aids in identifying prevalent entries which may represent common events or states in the system logs.

B.6 Missing Values Inspection

The analysis then checks for missing values across all columns, identifying columns with any missing entries. This step is crucial for assessing data quality and deciding if imputation or other data-cleaning processes are necessary prior to further analysis.

B.7 Unique Event ID Examination

For further insight, the code counts the number of unique 'EventId' entries if the column exists. This count is indicative of the diversity of events captured in the dataset, which may reflect the complexity and range of operations logged within the system.

Created by data-to-paper (AI)

B.8 Log Level Distribution Analysis

Finally, the distribution of log level entries is analyzed. This involves counting occurrences of different logging levels such as INFO, DEBUG, ERROR, etc. This distribution can provide an understanding of the system's operational status and alert prevalence over the logged period.

Throughout these steps, the extracted information is systematically written to a text file, enabling an organized record of the exploratory findings for further review and interpretation.

B.9 Code Output

data_exploration.txt

```
# Data Size
Number of rows: 2000, Number of columns: 11

# Summary Statistics
count        2000
mean    1.511e+04
std     1.139e+04
min          2931
25%          2931
50%         25746
75%         25746
max         25998

# Categorical Variables
## Logrecord
Logrecord
nova-api.log.1.2017-05-16_13:53:08          1060
nova-compute.log.1.2017-05-16_13:55:31       933
nova-scheduler.log.1.2017-05-16_13:53:08       7

## Date
Date
2017-05-16    2000

## Time
Time
00:05:21.242    3
00:12:05.112    3
00:11:33.093    3
00:06:43.298    3
00:05:41.236    2
00:00:31.094    2
00:11:33.092    2
```

Created by data-to-paper (AI)

```
00:08:06.962    2
00:08:06.961    2
00:08:06.960    2

## Level
Level
INFO       1969
WARNING      31

## Component
Component
nova.osapi_compute.wsgi.server                        809
nova.virt.libvirt.imagecache                          336
nova.compute.manager                                  262
nova.metadata.wsgi.server                             208
nova.compute.claims                                   168
nova.virt.libvirt.driver                              107
nova.compute.resource_tracker                          60
nova.api.openstack.compute.server_external_events      22
nova.api.openstack.wsgi                                21
nova.scheduler.host_manager                             7

## ADDR
ADDR
req-addc1839-2ed5-4778-b57e-5854eb7b8b09 - - - - -

    398
-

    155
req-3ea4052c-895d-4b64-9e2d-04d64c4d94ab - - - - -

    130
req-98474cd9-61e1-4afe-bd52-676a577b058f 113
    d3a99c3da401fbd62cc2caa5b96d2 54fadb412c4e40cdbaed9335e4c35a9e
     - - -     12
req-afb5ee70-39d1-435e-9bf9-2fc3339c5b5b 113
    d3a99c3da401fbd62cc2caa5b96d2 54fadb412c4e40cdbaed9335e4c35a9e
     - - -     12
req-d82fab16-60f8-4c9f-bde8-f362f57bdd40 113
    d3a99c3da401fbd62cc2caa5b96d2 54fadb412c4e40cdbaed9335e4c35a9e
     - - -     12
req-868a5460-dbb6-416b-b4c4-a98abae6c847 113
    d3a99c3da401fbd62cc2caa5b96d2 54fadb412c4e40cdbaed9335e4c35a9e
     - - -     12
req-72b4858f-049e-49e1-b31e-b562c5018eaf 113
    d3a99c3da401fbd62cc2caa5b96d2 54fadb412c4e40cdbaed9335e4c35a9e
     - - -     12
req-01d570b0-78a7-4719-b7a3-429fd7dc5a3f 113
```

Created by data-to-paper (AI)

```
    d3a99c3da401fbd62cc2caa5b96d2 54fadb412c4e40cdbaed9335e4c35a9e
     - - -     12
req-a07ac654-8e81-416d-bfbb-189116b07969 113
    d3a99c3da401fbd62cc2caa5b96d2 54fadb412c4e40cdbaed9335e4c35a9e
     - - -     12

## Content
Content
image 0673dd71-34c5-4fbb-86c4-40623fbe45b4 at (/var/lib/nova/
    instances/_base/a489c868f0c37da93b76227c91bb03908ac0e742): in
    use: on this node 1 local, 0 on other nodes sharing this
    instance storage    82
image 0673dd71-34c5-4fbb-86c4-40623fbe45b4 at (/var/lib/nova/
    instances/_base/a489c868f0c37da93b76227c91bb03908ac0e742):
    checking

    82
Active base files: /var/lib/nova/instances/_base/
    a489c868f0c37da93b76227c91bb03908ac0e742

    82
Unknown base file: /var/lib/nova/instances/_base/
    a489c868f0c37da93b76227c91bb03908ac0e742

    30
Removable base files: /var/lib/nova/instances/_base/
    a489c868f0c37da93b76227c91bb03908ac0e742

    30
Removing base or swap file: /var/lib/nova/instances/_base/
    a489c868f0c37da93b76227c91bb03908ac0e742

    21
HTTP exception thrown: No instances found for any event

    21
Compute_service record updated for cp-1.slowvm1.tcloud-pg0.utah.
    cloudlab.us:cp-1.slowvm1.tcloud-pg0.utah.cloudlab.us

    15
Auditing locally available compute resources for node cp-1.slowvm1
    .tcloud-pg0.utah.cloudlab.us

    15
Total usable vcpus: 16, total allocated vcpus: 1

    9

## EventId
```

Created by data-to-paper (AI)

```
EventId
E25    931
E34     82
E35     82
E27     82
E26     64
E21     44
E7      43
E36     30
E42     30
E8      22

## EventTemplate
EventTemplate
<*> "GET <*>" status: <*> len: <*> time: <*>.<*>
                                                                931
image <*> at (<*>): checking

    82
image <*> at (<*>): in use: on this node <*> local, <*> on other
    nodes sharing this instance storage     82
Active base files: <*>

    82
<*> "POST <*>" status: <*> len: <*> time: <*>.<*>
                                                                64
[instance: <*>] VM Resumed (Lifecycle Event)
                                                                     44
[instance: <*>] During sync_power_state the instance has a pending
     task (spawning). Skip.                43
Removable base files: <*>

    30
Unknown base file: <*>

    30
[instance: <*>] Instance destroyed successfully.
                                                                 22

# Missing Values
# Not Applicable

# Unique EventIds
Number of unique EventIds: 43

# Distribution of Log Levels
Level
INFO       1969
WARNING      31
```

Created by data-to-paper (AI)

C Data Analysis

C.1 Code

The Data Analysis was carried out using the following custom code:

```
# IMPORT
from my_utils import df_to_latex, df_to_figure
import pandas as pd
import numpy as np
from sklearn.utils import resample
import statsmodels.api as sm
import statsmodels.formula.api as smf
import pickle

# LOAD DATA
data = pd.read_csv('OpenStack_2k.log_structured.csv')

# DATASET PREPARATIONS
# No dataset preparations are needed.

# DESCRIPTIVE STATISTICS
## Figure df_desc_stat:
caption = "Most frequent EventTemplates by Log Levels"
# Aggregate and find the top EventTemplates
event_template_stats = data.groupby(['EventTemplate', 'Level']).
    ↪ size().unstack(fill_value=0)
most_frequent_templates = event_template_stats.sum(axis=1).
    ↪ nlargest(10).index
filtered_stats = event_template_stats.loc[most_frequent_templates]

# Plot the figure
df_to_figure(filtered_stats, 'df_desc_stat', caption=caption, kind
    ↪ ='bar', y=filtered_stats.columns.tolist())

# PREPROCESSING
# Creating a binary variable for log levels
data['Log_Level_Binary'] = data['Level'].apply(lambda x: 1 if x ==
    ↪  'WARNING' else 0)

# Simplifying EventTemplate to reduce dimensionality issues
data['EventTemplate_Simplified'] = data['EventTemplate'].str.
    ↪ extract(r'(\b[A-Z]+\b)', expand=False).fillna('Other')

# Creating a feature: Number of unique EventIds
unique_event_ids = data.groupby('EventTemplate_Simplified')['
    ↪ EventId'].transform('nunique')
data['Unique_EventIds'] = unique_event_ids
```

Created by data-to-paper (AI)

```
# Remove categories with few instances to avoid singular matrix
    ↪ issue
frequent_templates = data['EventTemplate_Simplified'].value_counts
    ↪ ()[lambda x: x > 10].index
balanced_data = data[data['EventTemplate_Simplified'].isin(
    ↪ frequent_templates)]

# ANALYSIS

## Table df_eventtemplate_warning:
caption = "Logistic regression analysis of simplified
    ↪ EventTemplate frequency relation to WARNING log levels"
# Resample to handle imbalanced data
info_data = balanced_data[balanced_data['Log_Level_Binary'] == 0]
warning_data = balanced_data[balanced_data['Log_Level_Binary'] ==
    ↪ 1]
warning_upsample = resample(warning_data, replace=True, n_samples=
    ↪ len(info_data), random_state=42)
balanced_resampled_data = pd.concat([info_data, warning_upsample])

# Logistic Regression
model = smf.logit(formula='Log_Level_Binary ~ C(
    ↪ EventTemplate_Simplified)', data=balanced_resampled_data).
    ↪ fit()

# Extracting details and ensuring no duplication in coefficient
    ↪ values
summary_df = pd.DataFrame({
    'coef': model.params,
    'ci_lower': model.conf_int()[0],
    'ci_upper': model.conf_int()[1],
    'p_value': model.pvalues
}).drop_duplicates()
summary_df = summary_df.head(20)  # Limit to 20 rows
df_to_latex(summary_df, 'df_eventtemplate_warning', caption=
    ↪ caption)

## Table df_eventtemplate_interaction:
caption = "Interaction analysis between simplified EventTemplate
    ↪ frequency and the number of unique EventIds"
# Logistic regression with interaction
interaction_model = smf.logit(formula='Log_Level_Binary ~ C(
    ↪ EventTemplate_Simplified) + Unique_EventIds', data=
    ↪ balanced_resampled_data).fit()

# Extracting interaction details and ensuring no duplication in
    ↪ coefficient values
interaction_summary_df = pd.DataFrame({
```

Created by data-to-paper (AI)

```
    'coef': interaction_model.params,
    'ci_lower': interaction_model.conf_int()[0],
    'ci_upper': interaction_model.conf_int()[1],
    'p_value': interaction_model.pvalues
}).drop_duplicates()
interaction_summary_df = interaction_summary_df.head(20)  # Limit
    ↪ to 20 rows
df_to_latex(interaction_summary_df, 'df_eventtemplate_interaction'
    ↪ , caption=caption)

# SAVE ADDITIONAL RESULTS
additional_results = {
    'Total number of observations': len(data),
    'Number of INFO levels': sum(data['Log_Level_Binary'] == 0),
    'Number of WARNING levels': sum(data['Log_Level_Binary'] == 1)
        ↪ ,
    'Unique EventTemplates': data['EventTemplate'].nunique(),
    'Unique EventIds': data['EventId'].nunique(),
}

with open('additional_results.pkl', 'wb') as f:
    pickle.dump(additional_results, f)
```

C.2 Provided Code

The code above is using the following provided functions:

```
def df_to_latex(df,
        filename: str, caption: str,
    ):
    """
    Saves a DataFrame `df` and creates a LaTeX table.
    `filename`, `caption`: as in `df.to_latex`.
    """


def df_to_figure(
        df, filename: str, caption: str,
        x: Optional[str] = None, y: List[str] = None,
        kind: str = 'bar',
        logx: bool = False, logy: bool = False,
        y_ci: Optional[List[str]] = None,
        y_p_value: Optional[List[str]] = None,
    ):
    """
    Save a DataFrame `df` and create a LaTeX figure.
    Parameters, for LaTex embedding of the figure:
    `filename`: Filename for the figure.
```

Created by data-to-paper (AI)

```
`caption`: Caption for the figure.

Parameters for df.plot():
`x`: Column name for x-axis (index by default).
`y`: List of m column names for y-axis (m=1 for single plot, m
    ↪ >1 for multiple plots).
`kind`: only bar is allowed.
`logx` / `logy` (bool): log scale for x/y axis.

`y_ci`: Confidence intervals for errorbars.
    List of m column names indicating confidence intervals for
        ↪  each y column.
    Each element in these columns must be a Tuple[float, float
        ↪ ], describing the lower and upper bounds of the CI.

 `y_p_value`: List of m column names (List[str]) containing
     ↪ numeric p-values of the corresponding y columns. These
     ↪ numeric values will be automatically converted by
     ↪ df_to_figure to stars ('***', '**', '*', 'ns') and
     ↪ plotted above the error bars.

If provided, the length of `y_ci`, and `y_p_value` should be
    ↪ the same as of `y`.

Example:
Suppose, we have:

df_lin_reg_longevity = pd.DataFrame({
    'adjusted_coef': [0.4, ...], 'adjusted_coef_ci': [(0.35,
        ↪ 0.47), ...], 'adjusted_coef_pval': [0.012, ...],
    'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
        ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
        ↪ [0.0001, ...],
}, index=['var1', ...])

then:
df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
    ↪ caption='Coefficients of ...', kind='bar',
    y=['adjusted_coef', 'unadjusted_coef'],
    y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
    y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval'])
"""
```

Created by data-to-paper (AI)

C.3 Code Description

C.4 Data Preparation and Descriptive Statistics

The analysis begins with aggregating and identifying the most frequent event templates categorized by log levels, such as INFO and WARNING. This step provides insights into which event templates are predominantly occurring and to identify any patterns in log severity levels. The top ten most frequent event templates were selected based on their cumulative occurrence across log levels for further analysis.

C.5 Data Preprocessing

To simplify and facilitate the analysis, a binary transformation of the log levels was created. Specifically, the log levels were binarized into a new variable 'Log_Level_Binary', where a level of 'WARNING' is coded as 1, and all other levels (notably 'INFO') are coded as 0. This transformation is crucial for logistic regression analysis where a binary outcome is necessary.

Furthermore, the dimensionality of the 'EventTemplate' was reduced by extracting and simplifying the template descriptions. This step minimizes potential issues linked to high dimensionality and sparsity, which can impair statistical model accuracy. If templates contained multiple descriptors, only the sequences of uppercase letters were retained, defaulting to 'Other' otherwise. Additionally, the number of unique 'EventIds' associated with each simplified template was computed and added as a new feature, providing a numeric indicator of the diversity of events within each template category.

To avoid singular matrix issues during regression analysis, event templates that did not appear frequently in the dataset (i.e., less than ten instances) were removed. This ensures the stability and reliability of statistical modeling.

C.6 Statistical Analysis

The statistical analysis is comprised mainly of logistic regression steps. To manage class imbalance between 'INFO' and 'WARNING' log levels, data resampling was applied. The 'WARNING' class data was upsampled to match the number of 'INFO' class instances using bootstrap sampling. This balanced dataset is employed in a logistic regression model exploring the relationship between the simplified event template frequencies and 'WARNING' log levels.

Created by data-to-paper (AI)

Following the primary analysis, a more detailed interaction analysis was conducted. This involved fitting a logistic regression model that examined interactions between the simplified event templates and the number of unique event IDs. Here, the aim is to understand how variability in event occurrences within templates might influence the likelihood of warning levels. This model thereby extends the analysis to consider the complexity and richness of events associated with different templates, rather than focusing solely on frequency.

C.7 Additional Results

Additional descriptive statistics and metadata about the dataset were computed and preserved for reference. This includes counts of observations, occurrences of each log level, and the number of unique event templates and event IDs. These results are valuable for documenting the scale and scope of the dataset and providing context for interpretation of the modeling outputs.

C.8 Code Output

df_desc_stat.pkl

```
Level

    INFO WARNING
EventTemplate
<*> "GET <*>" status: <*> len: <*> time: <*>.<*>
                                                                931
          0
Active base files: <*>

    82       0
image <*> at (<*>): checking

    82       0
image <*> at (<*>): in use: on this node <*> local, <*> on other
    nodes sharing this instance storage   82       0
<*> "POST <*>" status: <*> len: <*> time: <*>.<*>
                                                                64
    0
[instance: <*>] VM Resumed (Lifecycle Event)
                                                                     44
          0
[instance: <*>] During sync_power_state the instance has a pending
     task (spawning). Skip.              43       0
```

Created by data-to-paper (AI)

```
Removable base files: <*>

    30       0
Unknown base file: <*>

    0      30
<*> "DELETE <*>" status: <*> len: <*> time: <*>.<*>
                                                        22       0
```

df_eventtemplate_interaction.pkl

```
                                        coef   ci_lower  ci_upper
                                       p_value
Intercept                             -21.84        NaN       NaN
        nan
C(EventTemplate_Simplified)[T.GB]      2.295 -1.688e+04 1.688e+04
          1
C(EventTemplate_Simplified)[T.GET]   -0.1554 -1.627e+04 1.627e+04
          1
C(EventTemplate_Simplified)[T.HTTP]    2.295 -1.688e+04 1.688e+04
          1
C(EventTemplate_Simplified)[T.MB]      0.419        NaN       NaN
        nan
C(EventTemplate_Simplified)[T.Other]   1.057        NaN       NaN
        nan
C(EventTemplate_Simplified)[T.POST]   -2.388 -3.496e+04 3.496e+04
          1
C(EventTemplate_Simplified)[T.VCPU]    2.295 -1.688e+04 1.688e+04
          1
C(EventTemplate_Simplified)[T.VM]     -2.981        NaN       NaN
        nan
Unique_EventIds                       0.7264        NaN       NaN
        nan
```

df_eventtemplate_warning.pkl

```
                                        coef   ci_lower  ci_upper
                                       p_value
Intercept                             -23.28 -4.757e+04 4.752e+04
      0.999
C(EventTemplate_Simplified)[T.GB]    0.09519 -6.644e+04 6.644e+04
          1
C(EventTemplate_Simplified)[T.GET]    -5.143 -1.068e+05 1.068e+05
          1
C(EventTemplate_Simplified)[T.HTTP]  0.09519 -6.644e+04 6.644e+04
          1
C(EventTemplate_Simplified)[T.MB]    -0.9835 -6.612e+04 6.611e+04
          1
C(EventTemplate_Simplified)[T.Other]   24.29 -4.752e+04 4.757e+04
      0.999
```

Created by data-to-paper (AI)

```
C(EventTemplate_Simplified)[T.POST]  -0.9916   -6.6e+04 6.599e+04
          1
C(EventTemplate_Simplified)[T.VM]     -2.223 -8.046e+04 8.045e+04
          1
```

additional_results.pkl

```
{
    'Total number of observations': 2000,
    'Number of INFO levels': 1969,
    'Number of WARNING levels': 31,
    'Unique EventTemplates': 43,
    'Unique EventIds': 43,
}
```

D LaTeX Table Design

D.1 Code

The LaTeX Table Design was carried out using the following custom code:

```
# IMPORT
import pandas as pd
from my_utils import df_to_latex, df_to_figure, is_str_in_df,
    ↪ split_mapping, AbbrToNameDef

# PREPARATION FOR ALL TABLES AND FIGURES
all_mapping: AbbrToNameDef = {
    # Rename and provide glossary definitions for any abbreviated
        ↪ or not self-explanatory labels:
    'EventTemplate': ('Event Template', 'Template for event
        ↪ classification in logs'),
    'Log_Level_Binary': ('Log Level (Binary)', '1: WARNING level,
        ↪ 0: INFO level'),
    'Unique_EventIds': ('Unique Event IDs', 'Number of unique
        ↪ event identifiers for a given template'),
    'coef': ('Coefficient', 'Logistic regression coefficient'),
    'ci_lower': ('Lower CI', 'Lower bound of the 95% confidence
        ↪ interval'),
    'ci_upper': ('Upper CI', 'Upper bound of the 95% confidence
        ↪ interval'),
    'p_value': ('P-value', 'P-value of the coefficient\'s
        ↪ statistical significance'),
    'INFO': ('INFO', 'Log level indicating informational messages'
        ↪ ),
    'WARNING': ('WARNING', 'Log level indicating caution-worthy
        ↪ messages'),
    '<*> "GET <*>" status: <*> len: <*> time: <*>.<*>':
```

Created by data-to-paper (AI)

```
    ('GET Operation', 'General pattern for HTTP GET operations
        ↪ '),
'<*> "POST <*>" status: <*> len: <*> time: <*>.<*>':
    ('POST Operation', 'General pattern for HTTP POST
        ↪ operations'),
'[instance: <*>] VM Resumed (Lifecycle Event)':
    ('VM Resumed', 'Lifecycle event indicating a VM has
        ↪ resumed'),
'[instance: <*>] During sync_power_state the instance has a
    ↪ pending task (spawning). Skip.':
    ('Pending Task', 'Instance skipped during sync because it
        ↪ has a pending task (spawning)'),
'Unknown base file: <*>':
    ('Unknown Base File', 'Reference to a base file with
        ↪ unknown status'),
'<*> "DELETE <*>" status: <*> len: <*> time: <*>.<*>':
    ('DELETE Operation', 'General pattern for HTTP DELETE
        ↪ operations'),
'Active base files: <*>':
    ('Active Base Files', 'Files actively used in the current
        ↪ operation'),
'image <*> at (<*>): checking':
    ('Image Checking', 'Process checking an image at a
        ↪ specified location'),
'image <*> at (<*>): in use: on this node <*> local, <*> on
    ↪ other nodes sharing this instance storage':
    ('Image In Use', 'Image is in use on certain nodes'),
'Removable base files: <*>':
    ('Removable Base Files', 'Base files that can be removed
        ↪ without affecting operations'),

# Specific EventTemplate Simplifications
'C(EventTemplate_Simplified)[T.GB]': ('Event Template: GB', '
    ↪ General event template related to gigabytes'),
'C(EventTemplate_Simplified)[T.GET]': ('Event Template: GET',
    ↪ 'General event template related to HTTP GET operations')
    ↪ ,
'C(EventTemplate_Simplified)[T.HTTP]': ('Event Template: HTTP'
    ↪ , 'General event template related to HTTP operations'),
'C(EventTemplate_Simplified)[T.MB]': ('Event Template: MB', '
    ↪ General event template related to megabytes'),
'C(EventTemplate_Simplified)[T.Other]': ('Event Template:
    ↪ Other', 'General misc event template'),
'C(EventTemplate_Simplified)[T.POST]': ('Event Template: POST'
    ↪ , 'General event template related to HTTP POST
    ↪ operations'),
'C(EventTemplate_Simplified)[T.VCPU]': ('Event Template: VCPU'
    ↪ , 'General event template related to VCPU operations'),
```

Created by data-to-paper (AI)

```
    'C(EventTemplate_Simplified)[T.VM]': ('Event Template: VM', '
        ↪ General event template related to virtual machines'),
}

## Process df_desc_stat:
df_desc_stat = pd.read_pickle('df_desc_stat.pkl')

# Not Applicable

# Rename rows and columns:
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_desc_stat, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_desc_stat.rename(columns=abbrs_to_names, index=abbrs_to_names,
    ↪ inplace=True)

# Remove any stranded glossary keys
glossary.pop('Event Template', None)

# As a figure:
df_to_figure(
    df_desc_stat, 'df_desc_stat_formatted',
    caption="Most Frequent Event Templates by Log Levels",
    glossary=glossary,
    kind='bar',
    y=['INFO', 'WARNING'],
    xlabel="Event Template",
    ylabel="Frequency"
)

## Process df_eventtemplate_warning:
df_eventtemplate_warning = pd.read_pickle('
    ↪ df_eventtemplate_warning.pkl')

# Not Applicable

# Remove 'Intercept' column
df_eventtemplate_warning = df_eventtemplate_warning.drop(index='
    ↪ Intercept')

# Rename rows and columns:
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_eventtemplate_warning, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_eventtemplate_warning.rename(columns=abbrs_to_names, index=
    ↪ abbrs_to_names, inplace=True)

# As a table:
df_to_latex(
```

Created by data-to-paper (AI)

```
    df_eventtemplate_warning, 'df_eventtemplate_warning_formatted'
        ↪ ,
    caption="Logistic Regression Analysis of Simplified Event
        ↪ Template Frequency Relation to WARNING Log Levels",
    glossary=glossary)

## Process df_eventtemplate_interaction:
df_eventtemplate_interaction = pd.read_pickle('
    ↪ df_eventtemplate_interaction.pkl')

# Not Applicable

# Remove 'Intercept' column
df_eventtemplate_interaction = df_eventtemplate_interaction.drop(
    ↪ index='Intercept')

# Rename rows and columns:
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_eventtemplate_interaction, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_eventtemplate_interaction.rename(columns=abbrs_to_names, index=
    ↪ abbrs_to_names, inplace=True)

# As a table:
df_to_latex(
    df_eventtemplate_interaction, '
        ↪ df_eventtemplate_interaction_formatted',
    caption="Interaction Analysis Between Simplified Event
        ↪ Template Frequencies and the Number of Unique Event IDs"
        ↪ ,
    note="The table analyzes the interaction effects of different
        ↪ event templates with the unique event IDs on the
        ↪ likelihood of WARNING log levels.",
    glossary=glossary)
```

D.2 Provided Code

The code above is using the following provided functions:

```
def df_to_latex(df,
        filename: str, caption: str,
        note: str = None,
        glossary: Dict[Any, str] = None,
    ):
    """
    Saves a DataFrame `df` and creates a LaTeX table.
    `filename`, `caption`: as in `df.to_latex`.
    `note`: Note to be added below the table caption.
```

Created by data-to-paper (AI)

```
    `glossary`: Glossary for the table.
    """


def df_to_figure(
        df, filename: str, caption: str,
        note: str = None, glossary: Dict[Any, str] = None,
        x: Optional[str] = None, y: List[str] = None,
        kind: str = 'bar',
        logx: bool = False, logy: bool = False,
        y_ci: Optional[List[str]] = None,
        y_p_value: Optional[List[str]] = None,
        xlabel: str = None, ylabel: str = None,
    ):
    """
    Save a DataFrame `df` and create a LaTeX figure.
    Parameters, for LaTex embedding of the figure:
    `filename`: Filename for the figure.
    `caption`: Caption for the figure.
    `note`: Note to be added below the figure caption.
    `glossary`: Glossary for the figure.

    Parameters for df.plot():
    `x`: Column name for x-axis (index by default).
    `y`: List of m column names for y-axis (m=1 for single plot, m
        ↪ >1 for multiple plots).
    `kind`: only bar is allowed.
    `logx` / `logy` (bool): log scale for x/y axis.
    `xlabel`: Label for the x-axis.
    `ylabel`: Label for the y-axis.

    `y_ci`: Confidence intervals for errorbars.
        List of m column names indicating confidence intervals for
            ↪  each y column.
        Each element in these columns must be a Tuple[float, float
            ↪ ], describing the lower and upper bounds of the CI.

     `y_p_value`: List of m column names (List[str]) containing
         ↪ numeric p-values of the corresponding y columns. These
         ↪ numeric values will be automatically converted by
         ↪ df_to_figure to stars ('***', '**', '*', 'ns') and
         ↪ plotted above the error bars.

    If provided, the length of `y_ci`, and `y_p_value` should be
        ↪ the same as of `y`.

    Example:
    Suppose, we have:
```

Created by data-to-paper (AI)

```
    df_lin_reg_longevity = pd.DataFrame({
        'adjusted_coef': [0.4, ...], 'adjusted_coef_ci': [(0.35,
            ↪ 0.47), ...], 'adjusted_coef_pval': [0.012, ...],
        'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
            ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
            ↪ [0.0001, ...],
    }, index=['var1', ...])

    then:
    df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
        ↪ caption='Coefficients of ...', kind='bar',
        y=['adjusted_coef', 'unadjusted_coef'],
        y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
        y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval'])
    """


def is_str_in_df(df: pd.DataFrame, s: str):
    return any(s in level for level in getattr(df.index, 'levels',
        ↪  [df.index]) + getattr(df.columns, 'levels', [df.columns
        ↪ ]))

AbbrToNameDef = Dict[Any, Tuple[Optional[str], Optional[str]]]

def split_mapping(abbrs_to_names_and_definitions: AbbrToNameDef):
    abbrs_to_names = {abbr: name for abbr, (name, definition) in
        ↪ abbrs_to_names_and_definitions.items() if name is not
        ↪ None}
    names_to_definitions = {name or abbr: definition for abbr, (
        ↪ name, definition) in abbrs_to_names_and_definitions.
        ↪ items() if definition is not None}
    return abbrs_to_names, names_to_definitions
```

D.3 Code Output

df_desc_stat_formatted.pkl

```
\begin{figure}[htbp]
\centering
\includegraphics{df_desc_stat_formatted.png}
\caption{Most Frequent Event Templates by Log Levels
INFO: Log level indicating informational messages.
WARNING: Log level indicating caution-worthy messages.
GET Operation: General pattern for HTTP GET operations.
POST Operation: General pattern for HTTP POST operations.
VM Resumed: Lifecycle event indicating a VM has resumed.
Pending Task: Instance skipped during sync because it has a
    pending task (spawning).
```

Created by data-to-paper (AI)

```
Unknown Base File: Reference to a base file with unknown status.
DELETE Operation: General pattern for HTTP DELETE operations.
Active Base Files: Files actively used in the current operation.
Image Checking: Process checking an image at a specified location.
Image In Use: Image is in use on certain nodes.
Removable Base Files: Base files that can be removed without
    affecting operations.}
\label{figure:df-desc-stat-formatted}
\end{figure}
% This latex figure presents "df_desc_stat_formatted.png",
% which was created from the df:
%
% index,"INFO","WARNING"
% "GET Operation",931,0
% "Active Base Files",82,0
% "Image Checking",82,0
% "Image In Use",82,0
% "POST Operation",64,0
% "VM Resumed",44,0
% "Pending Task",43,0
% "Removable Base Files",30,0
% "Unknown Base File",0,30
% "DELETE Operation",22,0
%
% To create the figure, this df was plotted with the command:
%
% df.plot(kind='bar', y=['INFO', 'WARNING'], xlabel='Event
    Template', ylabel='Frequency')
```

df_eventtemplate_interaction_formatted.pkl

```
\begin{table}[h]
\caption{Interaction Analysis Between Simplified Event Template
    Frequencies and the Number of Unique Event IDs}
\label{table:df-eventtemplate-interaction-formatted}
\begin{threeparttable}
\renewcommand{\TPTminimum}{\linewidth}
\makebox[\linewidth]{%
\begin{tabular}{lrrrl}
\toprule
 & Coefficient & Lower CI & Upper CI & P-value \\
\midrule
\textbf{Event Template: GB} & 2.295 & -1.688e+04 & 1.688e+04 & 1
    \\
\textbf{Event Template: GET} & -0.1554 & -1.627e+04 & 1.627e+04 &
    1 \\
\textbf{Event Template: HTTP} & 2.295 & -1.688e+04 & 1.688e+04 & 1
     \\
```

Created by data-to-paper (AI)

```
\textbf{Event Template: MB} & 0.419 & NaN & NaN & nan \\
\textbf{Event Template: Other} & 1.057 & NaN & NaN & nan \\
\textbf{Event Template: POST} & -2.388 & -3.496e+04 & 3.496e+04 &
    1 \\
\textbf{Event Template: VCPU} & 2.295 & -1.688e+04 & 1.688e+04 & 1
     \\
\textbf{Event Template: VM} & -2.981 & NaN & NaN & nan \\
\textbf{Unique Event IDs} & 0.7264 & NaN & NaN & nan \\
\bottomrule
\end{tabular}}
\begin{tablenotes}
\footnotesize
\item The table analyzes the interaction effects of different
    event templates with the unique event IDs on the likelihood of
     WARNING log levels.
\item \textbf{Unique Event IDs}: Number of unique event
    identifiers for a given template
\item \textbf{Coefficient}: Logistic regression coefficient
\item \textbf{Lower CI}: Lower bound of the 95\% confidence
    interval
\item \textbf{Upper CI}: Upper bound of the 95\% confidence
    interval
\item \textbf{P-value}: P-value of the coefficient's statistical
    significance
\item \textbf{Event Template: GB}: General event template related
    to gigabytes
\item \textbf{Event Template: GET}: General event template related
     to HTTP GET operations
\item \textbf{Event Template: HTTP}: General event template
    related to HTTP operations
\item \textbf{Event Template: MB}: General event template related
    to megabytes
\item \textbf{Event Template: Other}: General misc event template
\item \textbf{Event Template: POST}: General event template
    related to HTTP POST operations
\item \textbf{Event Template: VCPU}: General event template
    related to VCPU operations
\item \textbf{Event Template: VM}: General event template related
    to virtual machines
\end{tablenotes}
\end{threeparttable}
\end{table}
```

df_eventtemplate_warning_formatted.pkl

```
\begin{table}[h]
\caption{Logistic Regression Analysis of Simplified Event Template
     Frequency Relation to WARNING Log Levels}
\label{table:df-eventtemplate-warning-formatted}
```

Created by data-to-paper (AI)

```
\begin{threeparttable}
\renewcommand{\TPTminimum}{\linewidth}
\makebox[\linewidth]{%
\begin{tabular}{lrrrl}
\toprule
 & Coefficient & Lower CI & Upper CI & P-value \\
\midrule
\textbf{Event Template: GB} & 0.09519 & -6.644e+04 & 6.644e+04 & 1
     \\
\textbf{Event Template: GET} & -5.143 & -1.068e+05 & 1.068e+05 & 1
     \\
\textbf{Event Template: HTTP} & 0.09519 & -6.644e+04 & 6.644e+04 &
     1 \\
\textbf{Event Template: MB} & -0.9835 & -6.612e+04 & 6.611e+04 & 1
     \\
\textbf{Event Template: Other} & 24.29 & -4.752e+04 & 4.757e+04 &
    0.999 \\
\textbf{Event Template: POST} & -0.9916 & -6.6e+04 & 6.599e+04 & 1
     \\
\textbf{Event Template: VM} & -2.223 & -8.046e+04 & 8.045e+04 & 1
    \\
\bottomrule
\end{tabular}}
\begin{tablenotes}
\footnotesize
\item \textbf{Coefficient}: Logistic regression coefficient
\item \textbf{Lower CI}: Lower bound of the 95\% confidence
    interval
\item \textbf{Upper CI}: Upper bound of the 95\% confidence
    interval
\item \textbf{P-value}: P-value of the coefficient's statistical
    significance
\item \textbf{Event Template: GB}: General event template related
    to gigabytes
\item \textbf{Event Template: GET}: General event template related
     to HTTP GET operations
\item \textbf{Event Template: HTTP}: General event template
    related to HTTP operations
\item \textbf{Event Template: MB}: General event template related
    to megabytes
\item \textbf{Event Template: Other}: General misc event template
\item \textbf{Event Template: POST}: General event template
    related to HTTP POST operations
\item \textbf{Event Template: VM}: General event template related
    to virtual machines
\end{tablenotes}
\end{threeparttable}
\end{table}
```